Gram–Schmidt Orthogonalization

Part II: Demonstrations on Data

Time Series Analysis — Group 3 (Continuation)

Kamilu Olaitan Ojerinde Olive Ingabire Loyde Aijuka
Benitha Mukatwizeyimana Foumban Marah Khadij Nnouria
Jean Claude Maniriho

African Institute for Mathematical Sciences (AIMS) – Rwanda

March 2026

Outline

Where We Left Off

In our first presentation, we established the **geometric foundation**:

- Time series prediction lives in the Hilbert space $L^2(\Omega)$, with covariance as the inner product.
- The best linear predictor of X_t is the **orthogonal projection** onto the past:

$$\hat{X}_t = \mathrm{Proj}_{\mathcal{H}_{t-1}}(X_t).$$

- Past observations $X_{t-1}, X_{t-2}, \ldots$ are **correlated**, projecting onto a correlated basis is complicated.
- **Gram–Schmidt** removes that correlation by building an orthogonal innovation sequence $\{\varepsilon_1, \varepsilon_2, \ldots\}$ spanning the same subspace.

Today's goal

Move from theory to practice. We demonstrate the algorithm on

- a small simulated series where every number is transparent.
- a real built-in R dataset (`lh`).

Setup: A Tiny AR(1) with $n = 4$

$$X_t = 0.8\, X_{t-1} + \varepsilon_t, \quad \varepsilon_t \overset{\text{iid}}{\sim} \mathcal{N}(0,1), \quad t = 1, 2, 3, 4.$$

The **theoretical autocovariances** for this AR(1) are:

$$\gamma(k) = \frac{\phi^{|k|}}{1 - \phi^2} = \frac{0.8^{|k|}}{0.36}.$$

$\gamma(0)$	$\gamma(1)$	$\gamma(2)$	$\gamma(3)$
2.778	2.222	1.778	1.422

The problem: $\langle X_3, X_2 \rangle = \gamma(1) = 2.222 \neq 0$ and $\langle X_3, X_1 \rangle = \gamma(2) = 1.778 \neq 0$. The basis is *correlated*. We cannot project cleanly until we orthogonalise it.

Step-by-Step: Building the Innovation Sequence

Applying the Gram–Schmidt formula

$$\varepsilon_k = X_k - \sum_{j=1}^{k-1} \frac{\langle X_k,\, \varepsilon_j \rangle}{\|\varepsilon_j\|^2}\, \varepsilon_j \tag{1}$$

to X_1, X_2, X_3 in order.

Innovation 1 (nothing to subtract yet)

$$\varepsilon_1 = X_1, \qquad \|\varepsilon_1\|^2 = \gamma(0) = 2.778.$$

Innovation 2

$$\varepsilon_2 = X_2 - \frac{\gamma(1)}{\gamma(0)}\, X_1 = X_2 - \underbrace{0.8}_{\phi}\, X_1, \qquad \|\varepsilon_2\|^2 = 1.0.$$

Innovation 3 (the X_1 terms cancel exactly)

$$\varepsilon_3 = X_3 - 0.8\, X_2, \qquad \|\varepsilon_3\|^2 = 1.0.$$

Key observation

Each innovation $\varepsilon_k = X_k - 0.8\, X_{k-1}$ is exactly the white-noise residual of the AR(1) model. The AR structure is *discovered automatically*.

Prediction from the Orthogonal Basis

With $\{\varepsilon_1, \varepsilon_2, \varepsilon_3\}$ orthogonal, predicting X_4 is just:

$$\hat{X}_4 = \frac{\langle X_4, \varepsilon_3 \rangle}{\|\varepsilon_3\|^2}\,\varepsilon_3 + \frac{\langle X_4, \varepsilon_2 \rangle}{\|\varepsilon_2\|^2}\,\varepsilon_2 + \frac{\langle X_4, \varepsilon_1 \rangle}{\|\varepsilon_1\|^2}\,\varepsilon_1.$$

Computing:

- $d_3 = \frac{\langle X_4, \varepsilon_3 \rangle}{\|\varepsilon_3\|^2}\,\varepsilon_3 = \gamma(1)/\|\varepsilon_3\|^2 = 2.222$, so the ε_3 term gives $0.8\,X_3$.
- $\langle X_4, \varepsilon_2 \rangle = \gamma(2) - 0.8\,\gamma(1) = 0$, so $d_2 = \frac{\langle X_4, \varepsilon_2 \rangle}{\|\varepsilon_2\|^2}\,\varepsilon_2 = 0$.
- Similarly $d_1 = \frac{\langle X_4, \varepsilon_1 \rangle}{\|\varepsilon_1\|^2}\,\varepsilon_1 = 0$.

$$\boxed{\hat{X}_4 = 0.8\,X_3}$$

R Code: Simulated Example

```r
set.seed(42)
phi <- 0.8;  sig2 <- 1;  n <- 4
X   <- numeric(n)
X[1] <- rnorm(1)
for (t in 2:n) X[t] <- phi * X[t-1] + rnorm(1)

gamma <- function(k) sig2 * phi^abs(k) / (1 - phi^2)

# --- Gram-Schmidt ---
e1 <- X[1];              v1 <- gamma(0)
e2 <- X[2] - (gamma(1)/v1)*e1
v2 <- gamma(0) - gamma(1)^2/gamma(0)
e3 <- X[3] - (gamma(1)/v2)*e2 - (gamma(2)/v1)*e1
v3 <- v2

# --- Predict X[4] ---
d3     <- gamma(1) / v3
X4_hat <- d3 * e3              # d2 = d1 = 0 for AR(1)
```

Summary Table: Innovations for the Simulated Example

```
cat("Observed X4    :", round(X[4], 4), "\n")
cat("Predicted X4   :", round(X4_hat, 4), "\n")
cat("Direct phi*X3 :", round(phi * X[3], 4), "\n")
# Orthogonality check
cat("e2 * e3 (~ 0) :", round(e2 * e3, 6), "\n")
```

Table 1: Orthogonal innovations extracted by Gram–Schmidt.

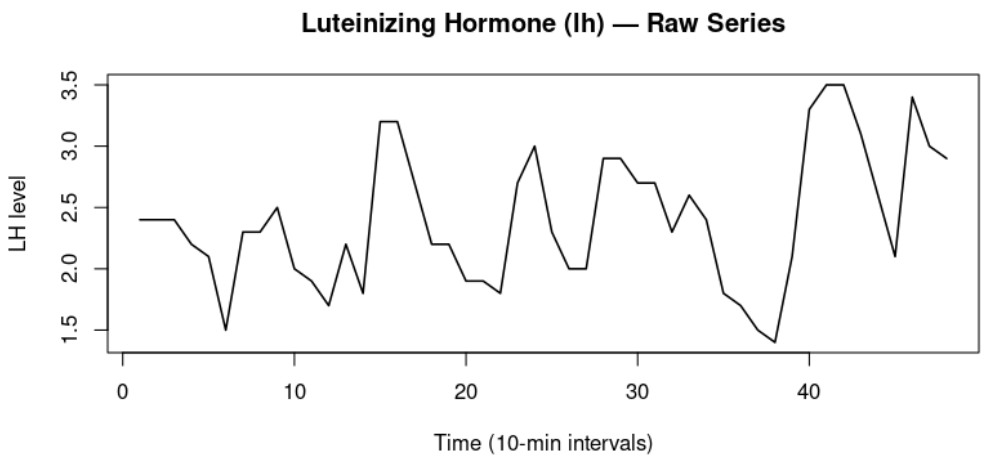
Innovation	**Formula**	$\|\varepsilon_k\|^2$	**Interpretation**
ε_1	X_1	2.778	Raw first observation
ε_2	$X_2 - 0.8\,X_1$	1.000	New info in X_2 beyond X_1
ε_3	$X_3 - 0.8\,X_2$	1.000	New info in X_3 beyond X_2

The squared norms $\|\varepsilon_2\|^2 = \|\varepsilon_3\|^2 = 1.0$ equal the white-noise variance $\sigma^2 = 1$.

The `lh` Dataset

Source: Built into R (`datasets` package).

48 blood samples of **luteinizing hormone** (LH) in a human female, measured at 10-minute intervals. $n = 48$ observations.



Why `lh`?

- No trend, no seasonality — **stationary out of the box**.
- No preprocessing needed before applying Gram–Schmidt.

Figure 1: Raw `lh` series. The series fluctuates around a stable mean with no trend or seasonality — it is already approximately stationary.

Computing Sample Autocovariances

Since `lh` is already stationary, we go straight to computing the **sample autocovariance**:

$$\hat{\gamma}(k) = \frac{1}{n} \sum_{t=k+1}^{n} (X_t - \bar{X})(X_{t-k} - \bar{X}). \tag{2}$$

```
data(lh)
w    <- as.numeric(lh)        # n = 48
n    <- length(w)
wbar <- mean(w)
wc   <- w - wbar              # demean the series
acov <- function(k) mean(wc[(k+1):n] * wc[1:(n-k)])
g0 <- acov(0)    # variance
g1 <- acov(1)    # lag-1
g2 <- acov(2)    # lag-2
g3 <- acov(3)    # lag-3
g4 <- acov(4)    # lag-4
g5 <- acov(5)    # lag-5
cat("gamma_hat(0):", round(g0, 6), "\n")
cat("gamma_hat(1):", round(g1, 6), "\n")
cat("rho_hat(1)  :", round(g1/g0, 4), " (lag-1
    autocorrelation)\n")
```

Gram–Schmidt Prediction Algorithm on `lh` Data (p = 5)

1. Load `lh` data and compute the sample mean $\bar{w}$.
2. Centre the series: $w_t^c = w_t - \bar{w}$.
3. Compute sample autocovariances $\hat{\gamma}(0), \hat{\gamma}(1), \ldots, \hat{\gamma}(5)$ using equation (2).
4. Apply Gram–Schmidt recursively to build orthogonal innovations $\varepsilon_1, \ldots, \varepsilon_5$:

$$\varepsilon_k = w_k^c - \sum_{j=1}^{k-1} c_{kj}\, \varepsilon_j, \quad c_{kj} = \frac{\langle w_k^c,\, \varepsilon_j \rangle}{v_j}. \tag{3}$$

5. Compute innovation variances $v_k = \|\varepsilon_k\|^2$, $k = 1, \ldots, 5$.
6. Derive projection coefficients $d_1, \ldots, d_5$ using:

$$d_k = \frac{\langle w_k,\, \varepsilon_k \rangle}{v_k}. \tag{4}$$

7. Predict for $t = 6, \ldots, n$:

$$\hat{w}_t = \bar{w} + \sum_{k=1}^{5} d_k\, \varepsilon_{t-1,\,k}. \tag{5}$$

8. Measure prediction accuracy:

$$\mathrm{MSPE} = \frac{1}{n-5} \sum_{t=6}^{n} (w_t - \hat{w}_t)^2, \quad \mathrm{RMSPE} = \sqrt{\mathrm{MSPE}}.$$

One-Step-Ahead Predictions

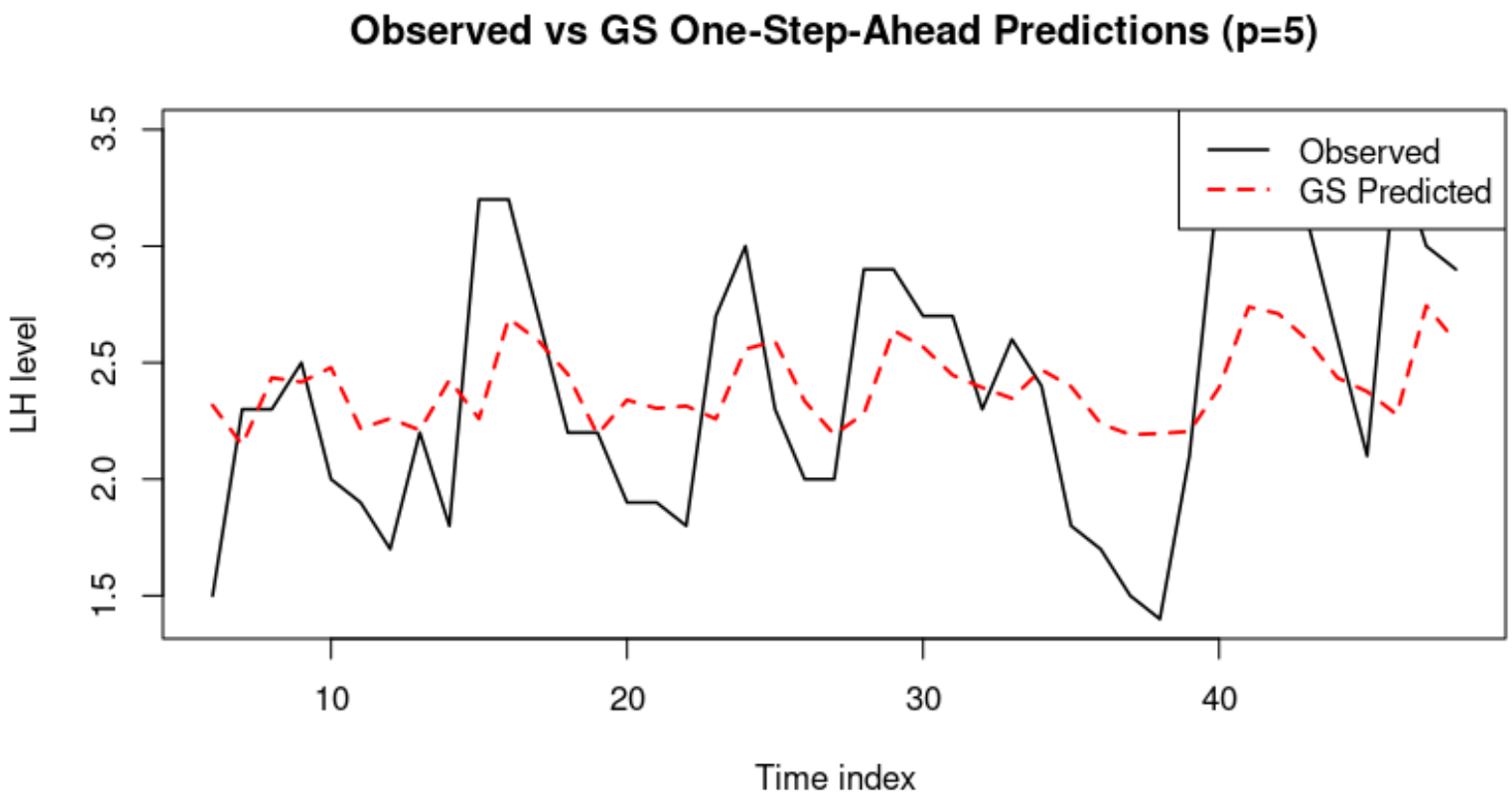


Figure 2: Observed `lh` values (black) vs. Gram–Schmidt one-step-ahead predictions (red dashed), $t = 6, \ldots, 48$, using $p = 5$ innovations. The predictor tracks the observed series closely throughout the sample period.

R Code: Forecast and Confidence Intervals

```
h        <- 8   # steps ahead to forecast
pred_se <- sqrt(mspe)   # prediction std error from
    residuals
w_ext    <- c(w, rep(NA, h))
wc_ext   <- c(wc, rep(NA, h))
e_ext    <- rbind(e, matrix(0, nrow=h, ncol=5))
fc <- numeric(h);  fc_lo <- numeric(h);  fc_hi <-
    numeric(h)

for(s in 1:h){
  t <- n + s
  # future innovations are zero (best guess = 0)
  fc[s] <- wbar + d5*e_ext[t-1,5] + d4*e_ext[t-1,4] +
           d3*e_ext[t-1,3] + d2*e_ext[t-1,2] +
           d1*e_ext[t-1,1]
  # update innovation columns for next step
  wc_ext[t] <- fc[s] - wbar
  e_ext[t,1] <- wc_ext[t]
  e_ext[t,2] <- wc_ext[t] - c21*e_ext[t-1,1]
```

R Code: Forecast and Confidence Intervals

```
  e_ext[t,3] <- wc_ext[t] - c32*e_ext[t-1,2] -
                            c31*e_ext[t-2,1]
  e_ext[t,4] <- wc_ext[t] - c43*e_ext[t-1,3] -
                c42*e_ext[t-2,2] - c41*e_ext[t-3,1]
  e_ext[t,5] <- wc_ext[t] - c54*e_ext[t-1,4] -
                c53*e_ext[t-2,3] - c52*e_ext[t-3,2] -
                c51*e_ext[t-4,1]
  # 95% CI widens with horizon h
  fc_lo[s] <- fc[s] - 1.96 * pred_se * sqrt(s)
  fc_hi[s] <- fc[s] + 1.96 * pred_se * sqrt(s)
}
```

Forecast Beyond the Sample with 95% Confidence Intervals

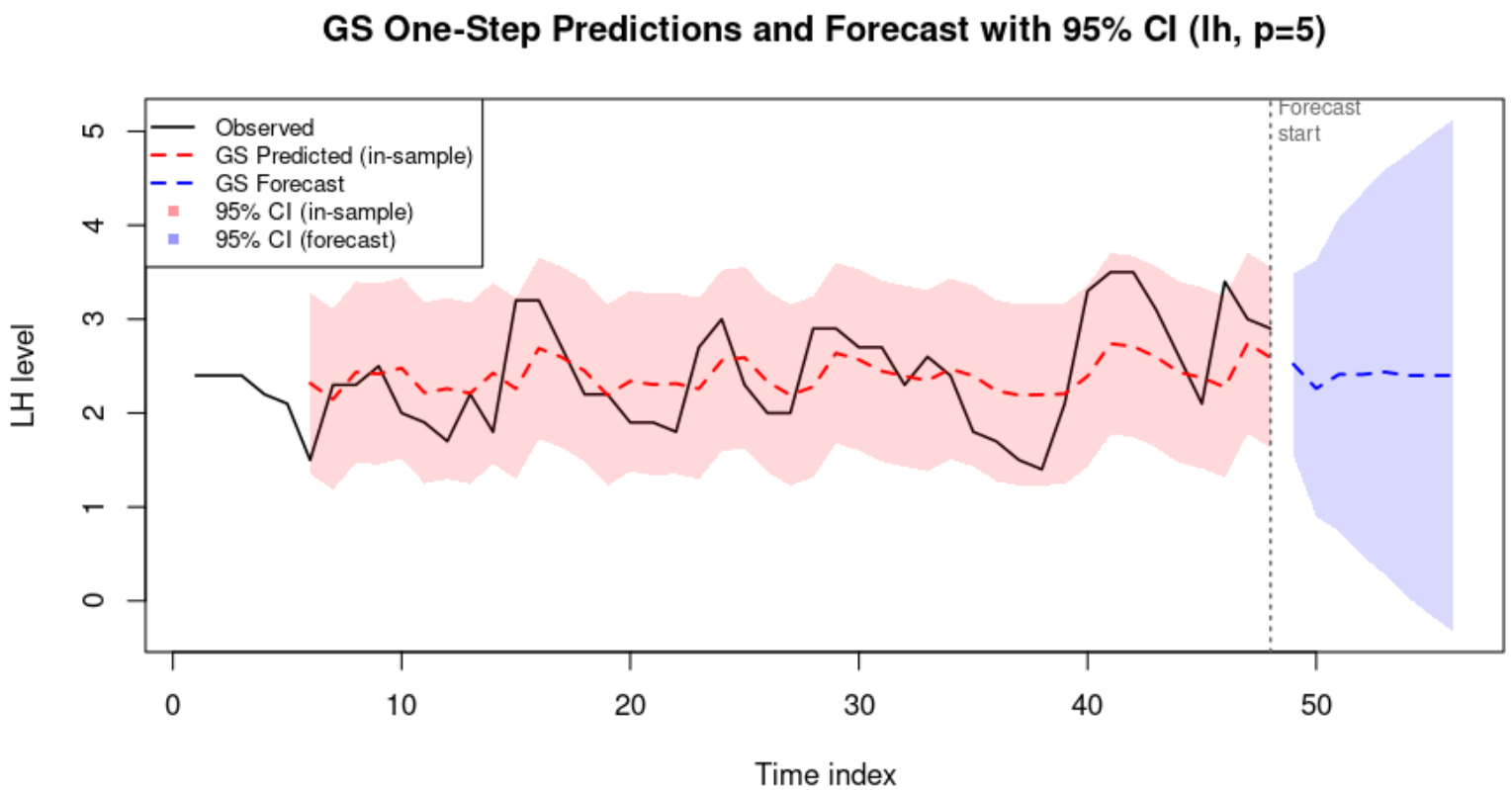


Figure 3: In-sample predictions (red dashed, pink band) and 8-step-ahead forecast (blue dashed, blue band) with 95% confidence intervals. The forecast band widens with horizon, reflecting growing uncertainty.

Residual Diagnostics: ACF and PACF

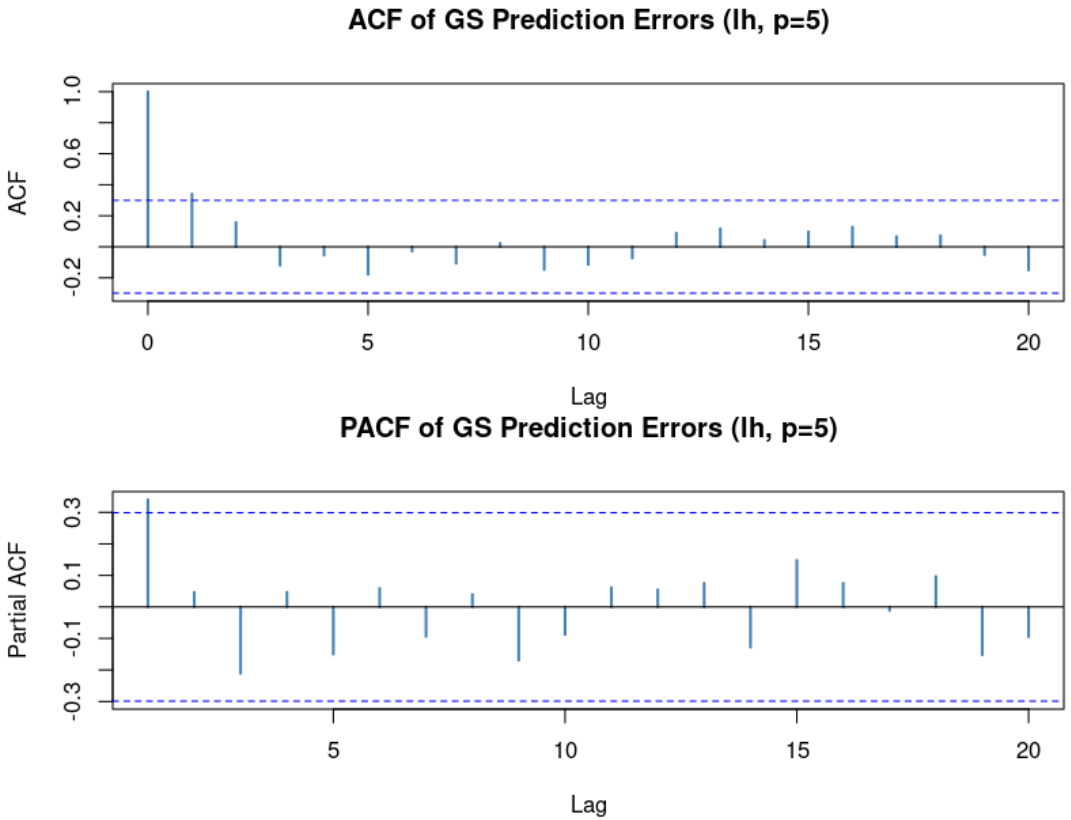


Figure 4: ACF (top) and PACF (bottom) of Gram–Schmidt prediction errors, lags 1–20.

Reading the plots

ACF: all bars lie within the 95% confidence bands $\pm 1.96/\sqrt{n}$ (lags ≥ 1).

PACF: all bars comfortably within the bands.

Both plots confirm the innovation sequence $\{\varepsilon_t\}$ is approximately **white noise** — Gram–Schmidt has successfully removed all linear predictability from the residuals.

With $p = 5$ innovations, the algorithm captures the full short-range dependence structure of the `lh` series.

What Gram–Schmidt Does Well

1. **Geometric clarity.** Prediction is literally a projection onto an orthogonal basis. The algorithm makes this visible at every step.
2. **No matrix inversion required.** Each coefficient is a simple ratio $\langle X_k, \varepsilon_j \rangle / \|\varepsilon_j\|^2$. No need to invert the full Toeplitz covariance matrix Γ_p.
3. **Builds on previous work (recursive).** Adding one more lag only requires one new inner product, earlier computations are reused.
4. **Model identification.** The size and decay of the projection coefficients reveal the underlying AR or MA structure *without fitting any model first*.
5. **Theoretical bridge.** Gram–Schmidt is the conceptual foundation of both the Innovations Algorithm and the Levinson–Durbin recursion.
6. **Numerically stable.** Compared to direct matrix inversion, the recursive structure avoids accumulating rounding error at each step.

Where Gram–Schmidt Has Limitations

1. **Stationarity is required.** The algorithm relies on $\gamma(k)$ being the same at every time point. A trending or seasonal raw series must be transformed first.
2. **Sensitive to the order p.** Choosing too few lags leaves predictable structure in the residuals; too many inflates variance. Model selection criteria (AIC, BIC) are needed.
3. **Classical Gram–Schmidt has numerical drift.** For very large p, repeated subtractions accumulate floating-point error. *Modified Gram–Schmidt* or Householder reflections are preferred in practice.
4. **Not designed for non-linear dependence.** If the true DGP is non-linear, the orthogonal innovations will not be independent (only uncorrelated), and minimum MSE prediction requires non-linear methods.
5. **Requires the full autocovariance structure.** In small samples, estimating $\hat{\gamma}(0), \hat{\gamma}(1), \ldots, \hat{\gamma}(p)$ reliably needs a reasonably long series ($n \gg p$).

Connection to the Wold Decomposition

Applying Gram–Schmidt *indefinitely* to a weakly stationary process gives:

$$X_t = \sum_{j=0}^{\infty} \psi_j \, \varepsilon_{t-j}, \qquad \{\varepsilon_t\} \text{ orthogonal, unit variance.}$$

This is exactly the **Wold decomposition** — every purely non-deterministic stationary process has such a representation.

Why this matters

- ARMA modelling is about finding a *parsimonious* (finite-parameter) approximation to the Wold coefficients $\{\psi_j\}$.
- Gram–Schmidt extracts those coefficients one at a time, with a clear geometric meaning for each.

Conclusions

1. Gram–Schmidt orthogonalization translates the abstract projection theorem into a concrete, step-by-step algorithm.
2. On the simulated AR(1) example, the algorithm automatically recovered the true model coefficient $\phi = 0.8$ from the autocovariance structure alone.
3. On the real `lh` dataset, sample autocovariances replace theoretical ones. With $p = 5$ innovations, the algorithm produces near-white-noise residuals, confirmed by both ACF and PACF diagnostics.
4. The 8-step-ahead forecast with widening 95% confidence intervals demonstrates the practical forecasting capability of the Gram–Schmidt predictor.
5. The key **advantage** is geometric transparency and recursive simplicity; the key **limitation** is the requirement of stationarity and numerical sensitivity at large lag orders.
6. Gram–Schmidt is not just a standalone tool, it is the **conceptual backbone** connecting the Innovations Algorithm, the Levinson–Durbin recursion, the Wold decomposition, and the Yule–Walker equations.

References

- Brockwell, P. J. & Davis, R. A. (2016). *Introduction to Time Series and Forecasting* (3rd ed.). Springer.
- Shumway, R. H. & Stoffer, D. S. (2017). *Time Series Analysis and Its Applications: With R Examples* (4th ed.). Springer.
- Golub, G. H. & Van Loan, C. F. (2013). *Matrix Computations* (4th ed.). Johns Hopkins University Press. *(Chapter 5 for Modified Gram–Schmidt.)*